

Adaptive Charging Threshold and Detour Scheduling for Wireless Rechargeable Sensor Networks via Deep Reinforcement Learning

Liang-Ching Tao, Wei-Cheng Tai, and Pi-Chung Wang*

Department of Computer Science and Engineering, National Chung Hsing University,
145 Xingda Rd., South District, Taichung 402, Taiwan
liangkingtao@gmail.com, allentai84@gmail.com, pcwang.tw@gmail.com

Abstract. On-demand charging lets a sensor request service when its energy falls below a threshold θ . Nearly all schemes fix one θ network-wide, ignoring the uneven relay load across clusters: too high and the charger queue saturates, too low and nodes die before service arrives. We add two cooperating deep reinforcement learning agents to a conventional charging stack: a threshold agent resets θ per cluster every 60s from its recent history, and a route agent decides at departure whether to detour to a nearby critical cluster. Against five schedulers across four system parameters, the framework matches or beats every baseline at 14 of 18 operating points and lifts default-setting survival from 78.0% to 92.2%. The spread matters more than the means: the fixed threshold collapses in 8 of 32 runs at 200 sensors; the adaptive one never falls below 86% over 60. The price is 3% more travel and longer waits.

Keywords: wireless rechargeable sensor networks, on-demand charging, deep reinforcement learning, adaptive threshold, detour scheduling

1 Introduction

Wireless sensor networks report over multi-hop trees to a base station (BS) [1], and the nodes nearest the BS relay traffic for everyone behind them, so they drain first and their loss disconnects whole subtrees. Wireless rechargeable sensor networks (WRSNs) address the battery limit by letting mobile chargers (MCs) replenish nodes in place through wireless power transfer [13, 12, 2]. Charging schemes divide into periodic and on-demand families [12]. Periodic schemes plan a fixed tour and are simple to run, but cannot react to the actual energy state of the field. On-demand schemes let a node issue a request once its residual energy ratio falls below a threshold θ , which is far more responsive [8, 15].

The responsiveness comes with a parameter that is rarely questioned. Almost all on-demand schemes fix a single θ , commonly 0.5, for the entire network [8, 21, 25]. This ignores the structural property just described: clusters near the BS differ systematically from those at the edge in how fast they drain and in

* Corresponding author.

how much warning a request gives. A single threshold therefore faces a dilemma we call the *fixed-threshold trap*. Set θ high and requests fire early and often, saturating the charger queue so that genuinely urgent nodes wait behind less urgent ones. Set it low and requests fire late, so a node can exhaust its energy before any charger arrives.

A second rigidity lies in dispatch. Conventional schemes send a charger on a strict base–target–base trip [8, 23]. A cluster that becomes critical while the charger is already en route, and that lies close to the path, cannot be served until the next dispatch cycle, even though the detour would have been short. Both rigidities are decisions, when to ask for service and whether to deviate, that a fixed rule cannot adapt.

We therefore add two cooperating deep reinforcement learning (DRL) agents to an otherwise conventional charging framework. A **threshold agent** adjusts θ per cluster from that cluster’s recent request history; a **route agent** decides at departure whether the charger should detour to a critical cluster on the way. The underlying clustering, routing and dispatch machinery is unchanged, so any difference in outcome is attributable to the two decisions we learn, and the fixed-threshold scheme the framework is built on is at the same time its exact ablation.

Our contributions are four. (i) We formulate per-cluster threshold adaptation as a partially observable control problem and solve it with a recurrent Q-network, so that the threshold reflects a cluster’s own history rather than a global constant. (ii) We add a route-level detour decision, restricted by a geometric corridor and trained in a second stage against the frozen threshold policy. (iii) We evaluate across four system parameters against five schedulers and report not only survival but travel, energy utilisation, request latency and charger occupancy, so that the cost of the gain is visible. (iv) We re-evaluate the main comparison on matched random seeds and report the full distribution rather than means alone, which reveals that the fixed-threshold baseline is bimodal: it collapses in roughly a quarter of runs at the density where the two schemes differ most.

2 Related Work

Periodic charging. Early work optimised the tour of a single charger. Xie et al. [24] showed that the optimal travel path is the shortest Hamiltonian cycle and established the renewable-energy framework; Lyu et al. [16] jointly optimised path and sojourn time with a hybrid particle-swarm and genetic search. With several chargers the problem acquires an assignment layer: Nguyen et al. [18] build multiple tours by ant-colony iteration to reduce the fleet, and Hong et al. [11] propose MCWP STO-CS, which clusters by k -means and plans spatiotemporally optimised routes; we use it as our periodic baseline. These schemes are stable under a fixed topology but cannot react to a sudden energy emergency.

On-demand charging and the threshold. On-demand charging replaces rigid tours with request-driven service [8], and refinements have added double warning

thresholds with pre-emption [15], nearest-job-next dispatch (NJNP) [8], fuzzy request prioritisation for several chargers [21], joint recharging and data collection [23], and game-theoretic bidding among chargers [22]. The fixed-threshold multi-charger scheme of Xu [25] is the framework this work extends. Across this line, the threshold that decides *whether* a request is issued remains a fixed global constant; it is the one parameter the literature has largely left alone, and it is what we make adaptive.

Learning-based charging schedulers. Reinforcement learning has entered the field in several forms. Li et al. [14] use a recurrent Q-network over node energy histories to decide the next service target in a charging-recycling scheme; Yang et al. [26] use an actor-critic to output the next stop and its sojourn time. Neither adapts the request threshold or considers a mid-trip detour. Value factorisation methods such as QMIX [19] target agents that must act jointly at the same time step; our two agents act at different granularities, one every 60 s per cluster and one once per dispatch, and are decoupled by staged training rather than by a joint value function. Ours is deliberately narrower than end-to-end dispatch learning: clustering, routing and queueing stay conventional, and learning is confined to two small decisions, which keeps the state space bounded and the learned behaviour easy to attribute.

3 System Model

3.1 Network Model

Fig. 1 shows the setting. N sensors are deployed uniformly at random over a 200×200 m field with the BS at the centre, and M mobile chargers start at the BS. Nodes report continuously and, because the communication range is 40 m, most reach the BS through relays. The routing tree is built once by Dijkstra's shortest path and rebuilt over the surviving nodes whenever a node dies. For node S_i we define its *relay count* r_i as the number of source-to-BS paths that pass through it, including its own. Nodes near the BS carry large r_i and drain far faster than the periphery; this heterogeneity is the reason a single network-wide threshold is a poor fit, and it is what the threshold agent's observation and reward are built around.

Each charger is in one of four states, idle, moving, charging or returning, and it is just before each departure that a decision point exists: whether to serve a critical cluster that lies along the way. The route agent of Section 4.2 acts there.

3.2 Charging Model

A charger carries an omnidirectional coil and charges every alive node within radius R_{charge} by magnetically coupled resonance [13]. The power received by S_i from charger MC_j follows the distance-dependent model of He et al. [9],

$$P_r(S_i, \text{MC}_j) = \frac{\alpha}{(d(S_i, \text{MC}_j) + \beta)^2} P_t, \quad d(S_i, \text{MC}_j) \leq R_{\text{charge}}, \quad (1)$$

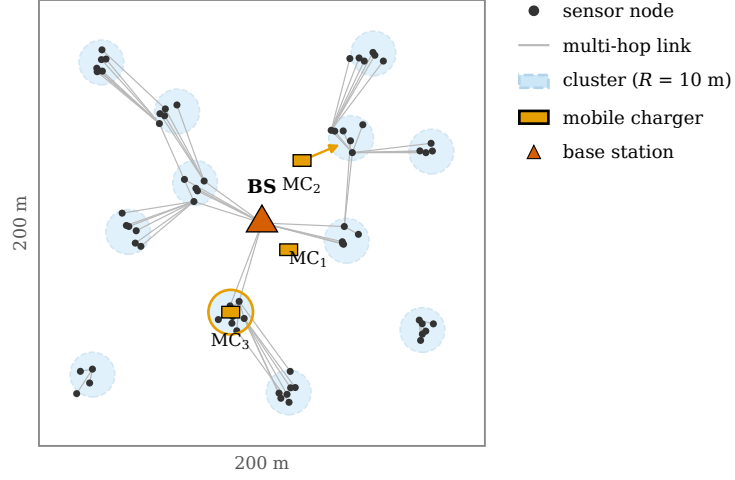


Fig. 1. System model. Sensors report over a multi-hop tree to the base station; nodes are grouped once by Mean-Shift into charging clusters of radius R ; mobile chargers are dispatched from the base station to cluster centres, where they park and charge every member within R . Each charger is at any time idle, moving, charging or returning.

where P_t is the transmit power and α, β are hardware and environment constants; $P_r = 0$ beyond R_{charge} . A node within reach of several parked chargers receives their sum, $P_r(S_i) = \sum_j P_r(S_i, MC_j)$. A charger executes a task by driving to the cluster centre, stopping, and charging every member in range until the task timer expires.

3.3 Energy Model

Nodes spend energy on sensing and computation, transmission and reception. Following the first-order radio model [10], and merging the first two into a constant P_{module} , the power drawn by S_i is

$$P_{S_i} = P_{\text{module}} + r_i f c_{\text{tx}} + (r_i - 1) f c_{\text{rx}}, \quad c_{\text{tx}} = \eta R_c^n, \quad (2)$$

where f is the per-node data rate, c_{rx} the per-bit reception cost, and c_{tx} the per-bit transmission cost, set by the attenuation coefficient η , the communication range R_c and the path-loss exponent n . The relay count multiplies both traffic terms, which is what makes the drain rate rise so steeply toward the BS.

3.4 Clustering, Requests and Dispatch

To exploit a charger's ability to serve several nodes at one stop, nodes are grouped once at initialisation by Mean-Shift clustering [5] with bandwidth $R = R_{\text{charge}} = 10$ m. Mean-Shift is preferred to k -means because it needs no preset

cluster count and is deterministic. A cluster is the unit at which requests are issued and served. The BS keeps a max-heap priority queue: when the residual energy ratio (state of charge, SoC) of any member of cluster g falls below the cluster's threshold θ_g , the cluster enters the queue with priority

$$\text{priority}(g) = \max_{n \in g} r_n, \quad (3)$$

so that the clusters holding critical relays, those draining fastest by (2), are served first; ties are broken first-come-first-served. At dispatch, the BS pops the head of the queue and assigns it to the available charger with the smallest estimated charging delay T_{cl} , computed from the charger's current state and position and subject to its keeping enough energy to complete the task and return. A popped cluster is locked to its charger and is not served twice.

This stack, clustering, routing, queueing and assignment, is inherited unchanged from the fixed-threshold scheme of Xu [25], in which $\theta_g = 0.5$ for every cluster and the charger always drives straight to its target. The proposed framework changes only two decisions inside it: the value of θ_g , and whether the charger deviates on the way. That is what makes the ablation of Section 5.5 exact.

4 Two Cooperating Agents

The framework places a Markov decision process, solved by value-based learning [20, 17], over the model of Section 3, with two agents at different granularities. The threshold agent runs every 60 s and sets θ_g for every cluster; the route agent runs once per dispatch and chooses between the direct trip and a detour. The loop is: (1) the threshold agent updates every θ_g ; (2) a cluster whose minimum SoC falls below θ_g issues a request; (3) the BS dispatches by (3) and Section 3.4; (4) at departure the route agent decides whether to detour. Table 1 lists the hyperparameters of both agents; every value below is the one used in the released code.

4.1 Threshold Agent

Observation. Every 60 s, for each cluster g , the agent forms a four-dimensional observation $s_g = [\overline{\text{SoC}}_g, \text{SoC}_{\min,g}, w_g, \bar{d}_g]$: the mean and minimum SoC over the cluster's members, a binary flag w_g set while the cluster is waiting in the queue, and the mean distance from the cluster centre to every charger that is not currently returning, normalised by the field diagonal, which serves as a proxy for expected service delay. All four are dynamic aggregates; cluster identity, coordinates and relay counts are deliberately excluded, so the policy is a state-to-action rule rather than a lookup on particular clusters. That choice is what lets one trained model be evaluated at other sensor counts, charger counts and speeds without retraining. Each dimension is normalised online by an exponential moving average of its mean and variance.

Action and smoothing. The action is one of 21 equally spaced thresholds $\theta \in [0.35, 0.60]$ (step 0.0125), which brackets the fixed value 0.5 of the baseline on both sides. A discrete action set with a DQN is preferred to continuous-action methods because the threshold does not need fine resolution and value-based training is more stable at this scale. The selected value is not applied directly but blended into the current one,

$$\theta_g \leftarrow (1 - \lambda) \theta_g + \lambda \theta_{\text{selected}}, \quad (4)$$

with λ stepping from 0.30 to 0.45 to 0.60 as the episode passes 10 ks and 60 ks, so that early, unconverged decisions cannot swing the threshold and later ones track the policy closely. One further rule guards against congestion: after 40% of the horizon, if more than 10% of clusters are waiting, the threshold of any cluster that is neither waiting nor below 20% SoC is lowered by up to 0.06, which relieves queue pressure without touching the clusters that are actually at risk.

Reward. The reward is global, computed once per update, and measures the last interval:

$$r_T = w_{\text{surv}} \Delta_{\text{alive}} - w_q \frac{N_w}{N_G} - w_{\text{wait}} f_{\text{overdue}} - w_{\text{bklg}} \frac{N_{\text{queue}}}{N_G} - w_{\text{death}} \mathbb{1}[\text{new death}], \quad (5)$$

where Δ_{alive} is the change in alive fraction since the previous update, N_w/N_G the fraction of clusters waiting, f_{overdue} the fraction of waiting clusters that have waited longer than $T_{\text{crit}} = 420\text{s}$, N_{queue}/N_G the queue length relative to the cluster count, and the last term a penalty whenever a node died in the interval. The weights are 5.0, 0.5, 0.30, 0.20 and 3.0. The maximum attainable reward is zero and corresponds to a network with nothing waiting and nothing lost; every term is a pressure the agent can relieve only by choosing thresholds well, since it controls nothing else.

Architecture. A single snapshot cannot tell a cluster that is draining quickly from one that has just been served, and a cluster’s demand depends on dispatch decisions made earlier; the problem is partially observable. We therefore use a deep recurrent Q-network (DRQN) [7]: an embedding layer (128) feeds a gated recurrent unit [4] (128) whose hidden state is kept per cluster across updates, and a linear head outputs the 21 action values. Training uses sequences of length $L = 16$ from a replay buffer of 20 000 sequences, double Q-learning [6] to limit overestimation, a target network updated softly with $\tau = 0.01$, and Adam with cosine-annealed learning rate. Exploration is ε -greedy, decaying from 0.60 to 0.10 over the first 1 800 decisions and capped at 0.04 after 40% of the horizon.

4.2 Route Agent

Candidate corridor. When a charger is dispatched to target cluster A , the agent may insert one detour to another cluster B before continuing to A (Fig. 2). To keep the action space bounded, candidates are first filtered by three rules and

then ranked. A cluster B qualifies if (i) its centre lies within $d_f \cdot R_{\text{charge}}$ of the segment from the charger to A , where d_f is the detour distance factor (default 3, swept in Section 5.2); (ii) it is urgent, $\text{margin}_B = \text{SoC}_{\min,B} - \theta_B < 0$ under the threshold of Section 4.1; and (iii) at least one of its members is alive. Qualifying clusters are scored by

$$\text{score}(B) = w_{\text{margin}} \text{margin}_B + w_{\text{cost}} \frac{\Delta d_B}{\ell}, \quad \Delta d_B = d_{\text{MC} \rightarrow B} + d_{B \rightarrow A} - d_{\text{MC} \rightarrow A}, \quad (6)$$

with $w_{\text{margin}} = 3$, $w_{\text{cost}} = 0.5$ and ℓ the field diagonal; the $K = 5$ lowest scores are kept, the most urgent and most on-path first. The filter only prunes; whether to detour at all, and to which of the K , is decided by the agent.

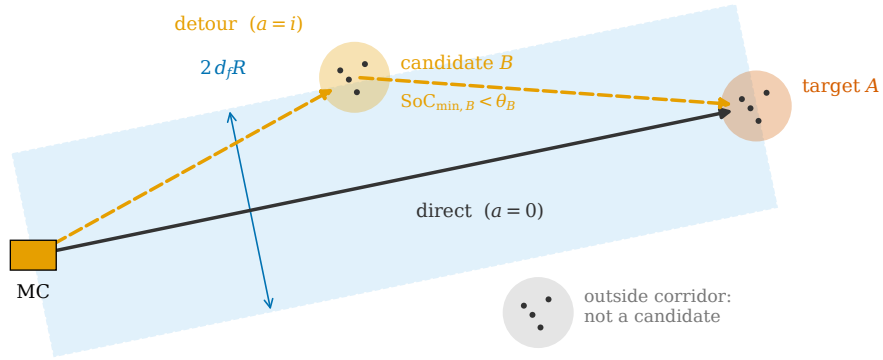


Fig. 2. Detour decision. The charger is bound for target A . A cluster B whose centre lies inside the corridor of half-width $d_f R$ around the direct segment, and whose minimum SoC is below its threshold, is a candidate; the agent chooses at departure between the direct trip ($a = 0$) and a detour through one candidate ($a = i$). Clusters outside the corridor are never candidates, however urgent.

Observation and action. The observation has 15 dimensions: five describing the trip, the normalised time slack of A (its minimum time-to-depletion minus the travel time, clipped), margin_A , the normalised distance to A , the normalised queue length and the number of qualifying candidates, followed by two per candidate, margin_{B_i} and $\Delta d_{B_i}/\ell$. Missing candidates are padded with a neutral pair (not urgent, very off-path) so the vector length is fixed. The action is one of $K + 1 = 6$: $a = 0$ drives straight to A ; $a = i$ visits candidate i first. The decision is a single macro-step made at departure and not revisited; in a multi-charger setting the candidate's state may change before arrival, and a re-planning variant is left for future work.

Reward. To avoid a sparse signal the trip is settled twice. On completing the detour leg at B the agent receives $r_{\text{mid}} = 0.5 - w_{\text{death}} \Delta_{\text{dead}}$, a small credit if no

node died on the way and a large penalty otherwise. On completing service at A it receives

$$r_D = w_{\text{time}} \left(-w_{\text{death}} \Delta_{\text{dead}} + \mathbb{I}[a > 0] (w_{\text{urg}} v_B - w_{\text{dist}} \Delta d_B^+) \right), \quad (7)$$

where Δ_{dead} counts deaths since the decision, v_B is the candidate’s urgency, $-\text{margin}_B/\theta_{\text{ref}}$ clipped to $[0, 1]$ with $\theta_{\text{ref}} = 0.20$, Δd_B^+ is the normalised extra distance, and $w_{\text{death}} = 10$, $w_{\text{urg}} = 2$, $w_{\text{dist}} = 0.3$. The distance weight is deliberately small so that cost never outweighs rescue. Decisions in the first 5 000 s carry $w_{\text{time}} = 2$, otherwise 1, because early losses propagate. The death penalty applies to direct trips and detours alike, so the agent is not rewarded for detouring as such, only for detours that save nodes.

Architecture. Because the decision is made once per dispatch from a compact vector, no recurrence is needed: a two-hidden-layer perceptron ($15 \rightarrow 64 \rightarrow 64 \rightarrow 6$) is the Q-network. Training uses uniform replay (5 000 transitions, batch 32), double Q-learning, a hard target copy every 200 steps, and ε decaying from 0.50 to 0.05 over 1 200 decisions, capped at 0.03 late in the episode. The macro-step reward is nearly a contextual bandit signal, which is why a small network suffices.

4.3 Two-Stage Curriculum

The two agents are coupled through the request stream: the threshold policy determines when and where requests appear, and that distribution *is* the environment the route agent learns in. Training both at once would let each agent’s environment drift with the other’s parameters, so neither could attribute an improvement in return to its own decisions. We therefore train in two stages (Algorithm 1), following the decoupling principle of curriculum learning [3]. In stage one the route agent is fixed to the direct trip and the threshold agent is trained alone, establishing a stable request distribution. In stage two the threshold agent is frozen with $\varepsilon = 0$ and the route agent is trained from scratch against it. Requests are triggered purely by SoC against threshold and are not directly coupled to charger paths; a detour changes when clusters are served, which affects SoC trajectories only as a second-order effect, and the threshold agent’s observation already carries w_g and $\bar{d}_g$ to absorb that. Empirically, training the route agent on the frozen threshold policy raises survival further (Section 5.5), which is evidence that the decoupling is sufficient for this problem. Joint fine-tuning after the two stages is left for future work.

5 Evaluation

5.1 Setup

All experiments run in a purpose-built discrete-time simulator over a horizon of 172 800 s (48 h). Table 2 lists the parameters, including the charging constants of (1); the default configuration is 200 sensors, three chargers at 5 m/s and 10 J

Algorithm 1 Two-stage curriculum training**Require:** simulator; hyperparameters of π_ϕ (threshold) and π_ψ (route)**Ensure:** trained parameters ϕ^*, ψ^* *Stage 1: threshold*

- 1: initialise ϕ ; fix π_ψ to the direct trip ($a \equiv 0$)
- 2: **for** each decision step until $t_{\text{sim}} \geq T_{\text{sim}}$ **do**
- 3: every 60 s, for each cluster: observe s_g , act ε -greedily by π_ϕ , apply (4); store sequence transitions
- 4: update ϕ by double DQN on length- L sequences; soft-update target
- 5: **end for**
- 6: $\phi^* \leftarrow \phi$

Stage 2: route

- 7: load π_{ϕ^*} and freeze it ($\varepsilon = 0$); initialise ψ
- 8: **for** each dispatch until $t_{\text{sim}} \geq T_{\text{sim}}$ **do**
- 9: build the corridor candidates, rank by (6), keep K
- 10: act ε -greedily by π_ψ ; settle r_{mid} and r_D by (7) as the trip completes
- 11: update ψ by double DQN; hard-copy target every 200 steps
- 12: **end for**
- 13: $\psi^* \leftarrow \psi$

Table 1. Hyperparameters of the two agents, as used in the released code.

	Threshold agent	Route agent
Network	Emb(128)–GRU(128)–FC(21)	FC(64)–FC(64)–FC(6)
Observation / actions	4 / 21	15 / 6
Decision interval	60 s per cluster	once per dispatch
Discount γ	0.99	0.95
Optimiser, LR	Adam, 3×10^{-4} , cosine-annealed	Adam, 3×10^{-4}
Batch / replay	64 / 20 000 seq., $L = 16$	32 / 5 000 transitions
ε schedule	$0.60 \rightarrow 0.10$, cap 0.04 late	$0.50 \rightarrow 0.05$, cap 0.03 late
Target network	soft, $\tau = 0.01$	hard copy / 200 steps
Reward weights	5.0, 0.5, 0.30, 0.20, 3.0 (5)	10, 2, 0.3 (7)
Structural constants	$T_{\text{crit}} = 420$ s; $\theta \in [0.35, 0.60]$	$d_f = 3$; $K = 5$; $\theta_{\text{ref}} = 0.20$

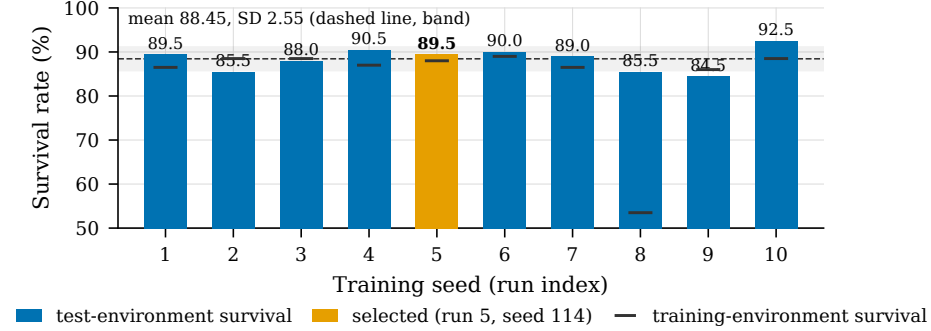
batteries. The simulator is deterministic given its seeds: node coordinates are fixed by an *environment seed* and initial energies, drawn in $[0.6, 0.8]$ of capacity, by a separate *energy seed*, so two methods run under the same pair face an identical network. Every experiment uses environment seed 1234; robustness to other topologies is not tested here and is listed as a limitation.

Training protocol. Stage one was repeated with ten independent DRL seeds on the default configuration. Fig. 3 shows the resulting test-environment survival: mean 88.5%, standard deviation 2.6 points, range 84.5–92.5%. Rather than the best seed, the model nearest the median (run 5, seed 114, 89.5%) was fixed as the input to stage two, so that reported gains are not inflated by seed selection; the spread of the ten runs is itself evidence of how stable stage one is. One seed

Table 2. Simulation parameters. Swept values are given as ranges with the default in bold.

Field, BS	200 × 200 m, centre	Sensors N	100–300 (200)
Horizon	172 800 s	Chargers M	1–6 (3)
Cluster radius R	10 m	Charger speed V_{mc}	3–7 m/s (5)
Comm. range R_c	40 m	Sensor battery E_s	10–50 J (10)
Initial SoC	60–80 %	Charger battery E_{mc}	117 000 J
Baseline θ	0.5	Transmit power P_t	3 W
Overdue T_{crit}	420 s	Charging α, β	36, 30
P_{module}	20 mW	Path-loss exponent n	4

(run 8) reached only 53.5% in its own training environment yet 85.5% at test, a reminder that training-environment survival is a poor selection criterion. Stage two was then repeated with three independent seeds (278, 279, 280) and all three route models were kept.

**Fig. 3.** Stage-one threshold agent over ten independent training seeds. Bars are test-environment survival; ticks are training-environment survival. The selected model (run 5) sits at the median, not the maximum.

Evaluation sets. Three sets of runs are used and we state for every figure which one it draws on. The *original sweep* evaluates every method at every point of the four parameter sweeps on energy seeds 1234–1236; the proposed method is run there with all three route models, nine runs per point, and the baselines with three. The *matched set* re-evaluates the sensor-count sweep on twelve energy seeds for both the fixed-threshold scheme and the proposed method, the latter with route model 278 alone, so that each point compares equal sample counts on identical networks; before merging, one previously published seed was re-run under our harness and reproduced the published survival rates exactly at all five sizes. At 200 sensors, where the two schemes differ most, the fixed-threshold scheme was extended to 32 seeds. All learned weights are frozen throughout;

the sweeps over charger count, speed and battery reuse the model trained at the default configuration, so they test robustness to system parameters, not generalisation to new deployments.

Compared methods. Five schedulers are compared, all implemented in the same simulator. **Fixed- θ** is the scheme this work builds on [25], with $\theta = 0.5$ and direct trips; with both agents disabled, our framework reduces to it. **MCWP STO-CS** [11] is periodic, with k -means clusters and precomputed tours. **Game Theory** [22] has chargers bid for requests on a payoff of chargeable energy minus travel cost and permits partial charging; **Modified Game Theory** forbids partial charging and uses our Mean-Shift clusters, isolating the bidding rule. **NJNP** [8] serves the nearest request next. We additionally report **DRL-Threshold**, our own framework with the route agent disabled, as the intermediate ablation; it is not counted as a baseline.

5.2 Sensitivity of the Route Agent

Two constants shape the route agent’s action space, and both were swept before the main comparison (Fig. 4). The corridor factor d_f was varied from 0 to 10, retraining the route agent at each value and evaluating twenty runs. Survival peaks at the default $d_f = 3$ with 93.0% and the tightest spread of the sweep (standard deviation 1.0 point); $d_f = 0$, which admits no candidates and reduces the agent to the direct trip, gives 88.3%. From $d_f = 4$ onward the spread exceeds 20 points at every value and the mean falls steadily, reaching 42–48% at $d_f \geq 8$, with the worst runs below 7%: a wide corridor lets the agent commit to detours that cost more than they save, and once a few relays are lost the network unravels. Corridor width is therefore not a tuning nicety but the mechanism that keeps detours affordable.

The candidate count K was varied from 1 to 10 with nine training runs each. Among runs that converged, survival is flat, 89.9–93.0% with the default $K = 5$ at the top and even $K = 1$ at 92.7%; the agent evidently uses the first-ranked candidate almost exclusively, which is what the ranking in (6) is designed to deliver. We report one detail that the summary hides: four of the ninety training runs, at $K = 3, 6$ and 9 , collapsed to survival below 7% and are excluded from the solid line. Training the route agent is not perfectly reliable, at a rate of about one run in twenty, and the collapses are not concentrated at any particular K . The three route models used in the main comparison all converged.

5.3 Survival across Four System Parameters

Fig. 5 reports survival along the four sweeps. Of the 18 distinct operating points, the proposed method is strictly ahead of every baseline at nine and tied with the best of them at five more, all at 100% survival where every threshold-based scheme saturates; the four points it loses are discussed in Section 5.7. Table 3 gives the sensor-count sweep numerically on the matched set of Section 5.1.

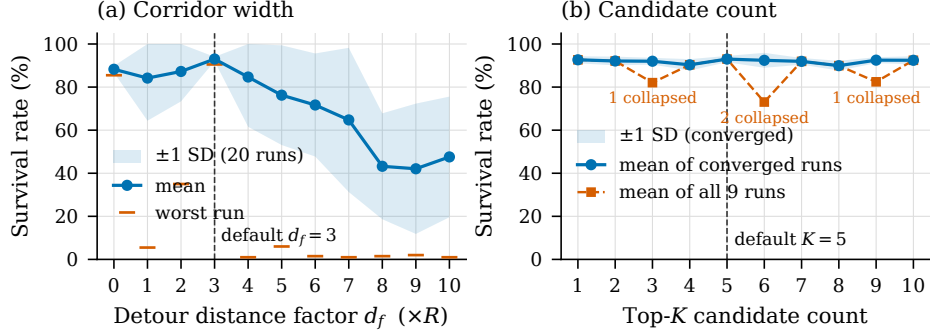


Fig. 4. Route-agent sensitivity at the default configuration. (a) Corridor factor d_f , twenty runs per value; the band is ± 1 standard deviation and the dashes mark the worst run. (b) Top- K candidate count, nine training runs per value (three energy seeds $\times$ three DRL seeds); the solid line averages the runs that converged, the dashed line averages all nine, and the labels give the number of training runs that collapsed below 50% survival.

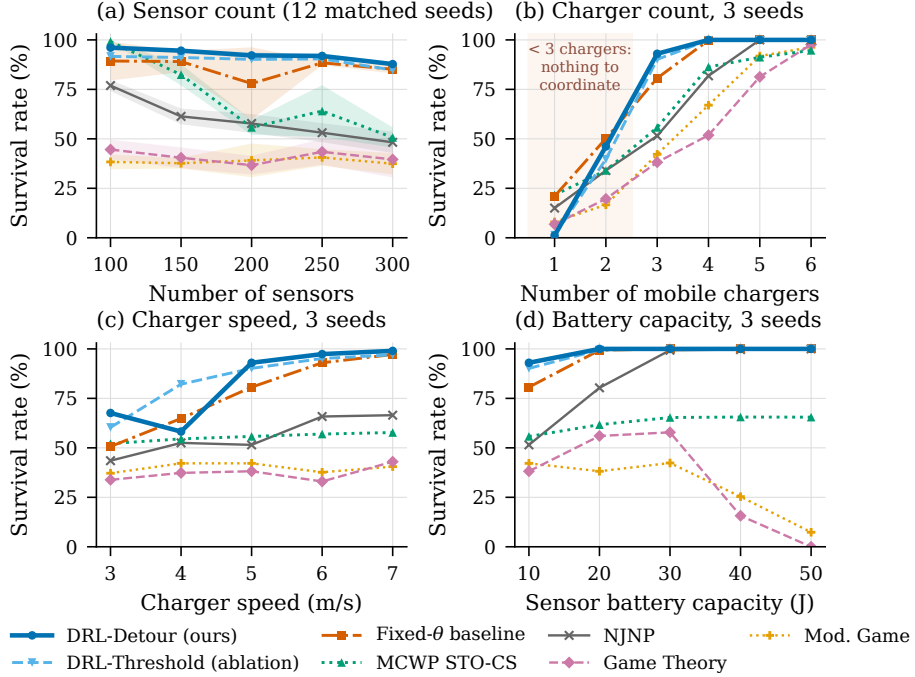


Fig. 5. Survival rate along four system parameters. (a) Sensor count on the matched set: twelve energy seeds per point, 32 for the fixed threshold at 200 sensors, bands ± 1 standard deviation; DRL-Threshold was not re-run and is drawn from the original three seeds without a band. (b)–(d) Charger count, charger speed and battery capacity on the original sweep, three energy seeds per point. The shaded region in (b) marks fleets of one or two chargers, where the framework has nothing to coordinate.

Table 3. Survival rate (%) versus sensor count on the matched set, mean $\pm$ standard deviation over n energy seeds. DRL-Detour uses route model 278.

Method	100	150	200	250	300
DRL-Detour ($n=12$)	96.1$\pm$1.0	94.6$\pm$0.9	92.2$\pm$1.6	91.9$\pm$1.0	87.8$\pm$1.0
Fixed- θ ($n=12$; 32 at 200)	89.3 $\pm$ 9.8	89.1 $\pm$ 5.4	78.0 $\pm$ 18.3	88.4 $\pm$ 1.2	85.2 $\pm$ 2.2
MCWP STO-CS ($n=15$)	99.2 $\pm$ 0.4	82.4 $\pm$ 5.4	55.8 $\pm$ 3.8	64.0 $\pm$ 13.1	50.7 $\pm$ 5.1
NJNP ($n=12$)	76.9 $\pm$ 2.8	61.3 $\pm$ 4.0	57.8 $\pm$ 4.8	53.1 $\pm$ 4.8	48.2 $\pm$ 5.4
Game Theory ($n=12$)	44.6 $\pm$ 5.0	40.4 $\pm$ 5.2	36.7 $\pm$ 4.7	43.4 $\pm$ 6.5	39.6 $\pm$ 9.1
Mod. Game Theory ($n=12$)	38.3 $\pm$ 3.7	37.7 $\pm$ 2.7	39.1 $\pm$ 8.5	40.6 $\pm$ 4.2	37.4 $\pm$ 5.4

Sensor count. As load rises from 100 to 300 sensors every method declines, but at very different rates. The proposed method falls from 96.1% to 87.8%, leading the fixed threshold by 2.6 to 14.2 points at every size and holding a standard deviation of at most 1.6 points throughout. The fixed threshold reaches a standard deviation of 18.3 points at 200 sensors; Section 5.4 shows why. MCWP STO-CS is the one method to beat ours, at 100 sensors with 99.2%: with so little load its fixed tour simply visits everything in time, but the tour lengthens with density and it collapses to 50.7% by 300. Two baselines have non-monotonic means. MCWP’s bump at 250 sensors is real: its 95% confidence interval there, 57–71%, only just touches the 54–58% at 200. The fixed threshold’s dip at 200 is instead the signature of a bimodal distribution rather than a trend. The game-theoretic pair stays near 40% at every size, because bidding on chargeable energy minus travel cost systematically favours near, half-empty clusters over distant, critical ones.

Charger count. Charger count is the sharpest axis. With one charger every method fails and ours fails worst, at 1.2% against 21% for the fixed threshold and MCWP; with two it trails the fixed threshold by four points. From three chargers the picture inverts: 93.0% against 80.7%, and from four upward every threshold-based scheme reaches 99.8–100%, while the game-theoretic schemes need six chargers to pass 95% and the periodic scheme does not reach it even then. The framework is a multi-charger coordination mechanism, and below three chargers there is nothing to coordinate; a detour by the only available charger merely delays its target.

Charger speed. Faster chargers help everyone, but the threshold-based schemes profit most: from 3 to 7 m/s the proposed method climbs from 67.6% to 99.0% and the fixed threshold from 50.7% to 97.2%, while MCWP stays between 52% and 58% because its tour is already the bottleneck. At the slowest speed the lead over the fixed threshold is 17 points, the largest on any axis. The exception is 4 m/s, where the full framework (58.2%) falls behind both the fixed threshold (64.8%) and its own threshold-only ablation (82.2%). The route agent was trained at 5 m/s, and at 4 m/s its learned notion of an affordable detour is systematically too generous: detours it would have completed in time at 5 m/s now

arrive late at the target. This is the one point on any axis where the route agent does net harm, and it is a speed-transfer failure rather than a scheduling one.

Battery capacity. Capacity is the axis on which the threshold-based schemes saturate first: at 20 J and above all three reach 99.3–100%, so only the 10 J point discriminates, and there the ordering is the default one. NJNP also saturates from 30 J. MCWP gains only ten points across the whole range, from 56% to 66%, since a fixed tour cannot exploit the slack a larger battery provides. The game-theoretic pair *deteriorates* as capacity grows, from 38–42% at 10 J to 0–7% at 50 J. We did not isolate the cause; the bidding rule already idles a third of the fleet at 10 J (Section 5.6), and with rarer requests it has still less reason to send a charger to a distant critical cluster.

5.4 The Fixed-Threshold Trap Is Bimodal

Averages understate what happens at 200 sensors. Fig. 6 plots every individual run. The fixed threshold is not merely noisier; it is bimodal. In 24 of 32 runs it behaves normally, with a median of 88%; in the remaining 8 it collapses, to values between 9% and 68%. The collapse rate is stable across three independently drawn batches of seeds: 1 of 3 in the originally published set, 2 of 9 in our first extension and 5 of 20 in our second, giving 8 of 32 overall, or 25% (95% confidence interval 10–40%). It is a property of the scheme, not a run of bad luck, and it is not confined to 200 sensors: over the 80 fixed-threshold runs of the matched set, one further run at 100 sensors also fell below 70%.

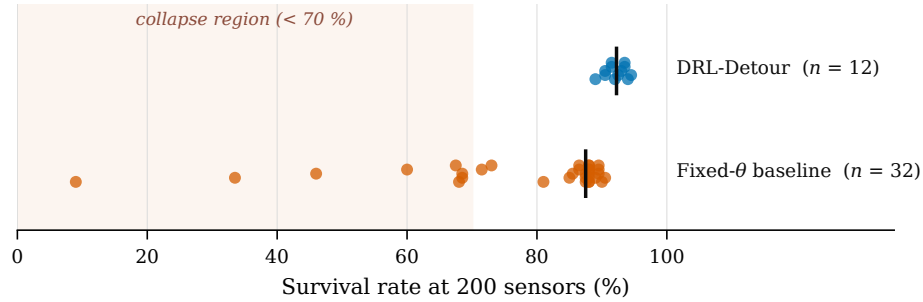


Fig. 6. Every individual run at 200 sensors on the matched set. Vertical bars mark medians. The fixed threshold is bimodal: 8 of 32 runs collapse, one to 9% survival. The adaptive threshold never enters the collapse region; its worst run over all 60 matched runs, at any sensor count, is 86.3%.

Under the same conditions the adaptive threshold never falls below 86.3% across the 60 runs spanning all five sizes. The mechanism it removes is the trap itself: when a single global θ happens to be mismatched to a particular initial-energy draw, requests either flood the queue or arrive too late, a critical relay

dies, its subtree loses its route, and the cascade is unrecoverable within the horizon. Adapting θ per cluster prevents the mismatch from becoming global. We note that adding seeds made the dip *deeper*, not shallower: the mean at 200 sensors fell from 82.2% at 12 seeds to 78.0% at 32, because the additional draws included five more collapses. That is the expected behaviour of a bimodal distribution and worth stating, since the intuition that more samples smooth a curve holds only when the distribution is unimodal.

5.5 What Each Agent Contributes

Because the two agents can be enabled independently, the decomposition is exact rather than inferred. Table 4 reports the three configurations on the three energy seeds and the single route model (278) for which every variant was evaluated, so the sample counts are equal.

Table 4. Ablation on matched seeds (survival rate, %). “Threshold only” disables the route agent; “Fixed θ ” disables both and reverts to $\theta = 0.5$.

Configuration	100	150	200	250	300
Both agents	96.3	94.9	92.7	92.1	87.2
Threshold only	91.7	91.1	90.2	90.4	84.8
Fixed $\theta = 0.5$	92.3	91.3	80.7	89.1	85.9
Threshold contributes	−0.7	−0.2	+9.5	+1.3	−1.1
Detour adds	+4.7	+3.8	+2.5	+1.7	+2.4

The decomposition is not what a headline average would suggest, and it is more informative for being uneven. The detour agent contributes consistently, adding between 1.7 and 4.7 points at every size. The threshold agent does not: it is worth +9.5 points at 200 sensors and roughly nothing, within noise and sometimes slightly negative, everywhere else. That pattern is coherent rather than anomalous. Adapting θ pays only where a fixed θ actually fails, and Section 5.4 showed that 200 sensors is precisely where it fails. At the other sizes $\theta = 0.5$ happens to suit the load, so there is nothing for adaptation to recover and the learned policy merely matches it. The value of threshold adaptation is therefore insurance against a failure mode, not a uniform gain, which is the same conclusion the distribution plot reaches by a different route. The two contributions stack: on the original sweep, averaged over all 21 points, the route agent adds 0.9 points on top of the threshold agent and the full framework leads the fixed threshold by 2.6.

5.6 What the Gain Costs

A scheduler can buy survival with travel, with charger energy, or with longer waits, and Figs. 7 and 8 show which. All cost figures are from the original sweep.

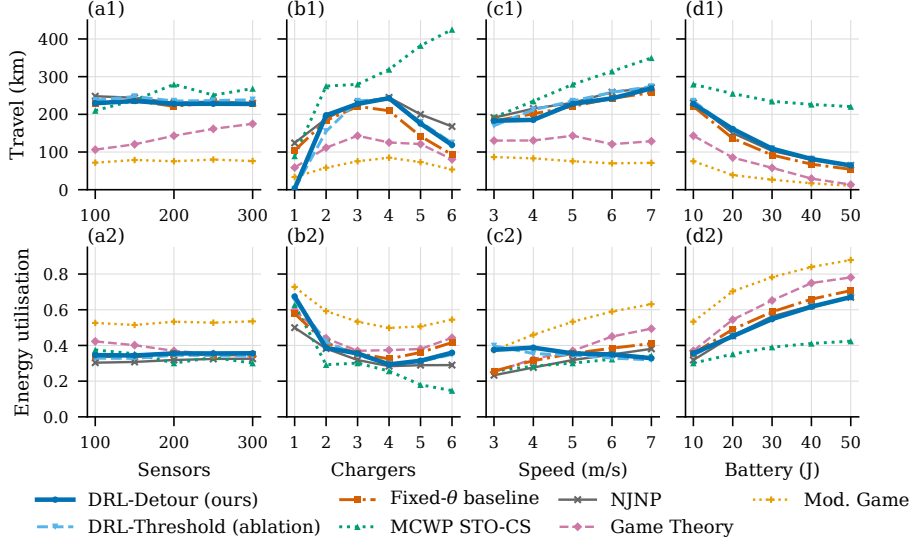


Fig. 7. Mean travel per charger (top row) and charger energy utilisation, the fraction of charger energy delivered as charge rather than spent moving (bottom row), along the four sweeps. Original sweep, three energy seeds. The threshold-based schemes are nearly indistinguishable on both metrics; the game-theoretic schemes travel least and utilise best because they serve least.

Travel and energy utilisation. At the default configuration the proposed method drives 228 km per charger over the horizon against 221 km for the fixed threshold, 3% more, and the threshold-only variant 235 km; energy utilisation is 0.355 for both ours and the fixed threshold. Along every axis the three threshold-based curves lie almost on top of one another. Detours are short by construction, and the extra distance they add is offset by the trips they save when the detoured cluster no longer needs a separate dispatch. MCWP travels at least as far as ours at every point beyond the lightest load, and up to 3.6 times as far once chargers are plentiful or batteries large, because its tour visits every cluster whether or not it needs charge; its utilisation is correspondingly the lowest. The game-theoretic schemes travel one-third to two-thirds as far and post the highest utilisation on every axis, but Fig. 5 shows what that buys: they travel less because they serve fewer clusters, and utilisation measured on the energy actually delivered says nothing about the energy that should have been. On the battery axis travel falls for every method as capacity grows, since requests become rarer; the ordering is unchanged.

Request latency. Here the cost is visible. At the default configuration a request waits a median of 157 s under the proposed method against 116 s under the fixed threshold, and the 90th percentile is 368 s against 318 s. The reason is that the adaptive threshold sits above 0.5 for the clusters that matter, so requests are

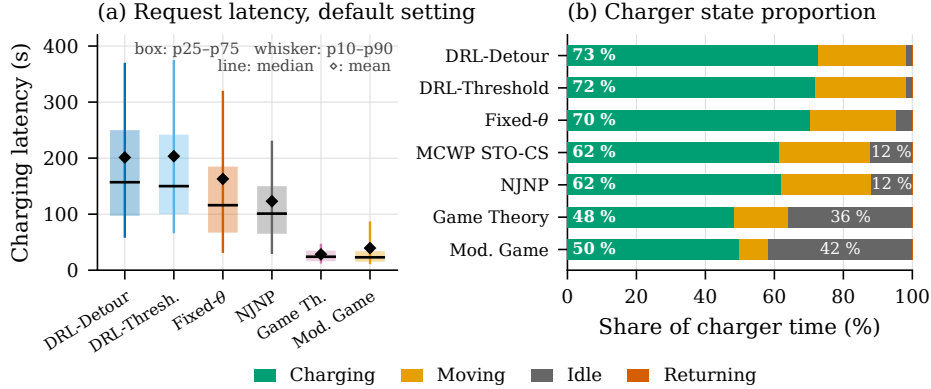


Fig. 8. (a) Distribution of request latency, the time from a cluster’s request to the start of its service, at the default configuration; MCWP STO-CS is periodic and issues no requests. (b) Share of the horizon each charger spends charging, moving, idle and returning at the default configuration.

issued earlier and more often: 7 260 per run against 6 300. More requests queue for the same three chargers, and each waits longer. The waits are, however, waits of nodes that still have charge, which is why the longer queue coexists with fewer deaths. The game-theoretic schemes show the mirror image, a median wait of 24s, because a request that will not be served promptly is simply outbid and never enters a queue at all. Along the charger axis latency falls for every method as chargers are added, and from four chargers the medians of the three threshold-based schemes are within ten seconds of one another.

Charger occupancy. Fig. 8(b) shows where the charger fleet’s time goes. Under the proposed method a charger spends 72.7% of the horizon charging, 25.5% moving and 1.7% idle; under the fixed threshold 70.5%, 24.9% and 4.5%. Adaptive thresholds convert idle time into charging time, which is the intended effect of issuing requests before the queue empties. The periodic and nearest-job schemes idle 12% of the time and the game-theoretic pair 36–42%, which is the direct cause of their survival: a fleet that is idle a third of the time while nodes are dying is not resource-limited but decision-limited.

5.7 Where the Framework Does Not Help

Four operating points go against the framework, each for a cause already visible in Section 5.3. At 100 sensors MCWP STO-CS wins, 99.2% against 96.1%, because the load is light enough for a fixed tour: a ceiling effect, not a failure. With one or two chargers the framework trails the fixed threshold by 20 and 4 points, a scope boundary: it coordinates a fleet and has nothing to coordinate below three. At 4 m/s the route agent, trained at 5 m/s, overcommits to detours and the full framework trails its own threshold-only variant by 24 points; this is

the one genuine weakness, and a deployment at a different charger speed should retrain stage two. On the remaining fourteen distinct points the framework is ahead or tied at 100%; its margin over the strongest baseline at the nine points it wins outright runs from 0.2 points, at four chargers where both are near saturation, to 15.5 points at 3 m/s, with 12.3 at the default setting.

5.8 Threats to Validity

Four limitations bound these results. The evaluation fixes the environment seed, so all runs share one node layout and vary only initial energy; the threshold agent’s identity-free observation is designed to transfer to other layouts, but that is untested. The four-parameter sweeps use three energy seeds, and only the sensor-count axis was re-evaluated on matched seeds, so the finer distinctions on the other three axes (for example the four-point gap at two chargers) are within the noise those sweeps can resolve. The route agent’s training is not perfectly reliable, collapsing in about one run in twenty across the Top- K sweep, so a deployment should verify the trained policy against the direct-trip baseline before use. And the matched comparison uses a single route model rather than averaging three, trading a small amount of variance reduction for exact seed alignment with the baselines; the original sweep, which averages three models, agrees with it at every shared point to within one point.

6 Conclusion

Making the charging threshold adaptive per cluster, and allowing a dispatched charger to detour to a nearby critical cluster, matches or improves on every baseline at 14 of 18 operating points across four system parameters, but the two mechanisms do different work. The detour agent contributes steadily, two to five points at every size; threshold adaptation pays where a fixed threshold fails, and removes a failure mode rather than improving an average. On matched seeds the framework leads at every network size, and at the density where the schemes differ most it eliminates a collapse that the fixed-threshold scheme suffers in a quarter of runs. The price is modest and specific: 3% more travel, no loss of charger energy utilisation, and longer request waits, which are the waits of nodes that still have charge. Two boundaries are equally specific: the framework needs at least three chargers to have anything to coordinate, and the route agent should be retrained if the charger speed changes.

Reporting the distribution rather than the mean was necessary to see the main result, and we suggest it as a default for charging-scheduler evaluation: where a scheduler can fail catastrophically, the mean is the wrong summary. Future work should vary the network topology as well as the initial energy, examine whether the threshold policy transfers across deployment densities without re-training, and replace the one-shot detour decision with one that can be revised en route.

Acknowledgements. This work was supported by the National Science and Technology Council, Taiwan, under Grant No. NSTC 114-2221-E-005-043-MY3. Parts of this work appeared in the second author's master's thesis.

References

1. Akyildiz, I.F., Su, W., Sankarasubramaniam, Y., Cayirci, E.: Wireless sensor networks: a survey. *Computer Networks* **38**(4), 393–422 (2002). DOI 10.1016/S1389-1286(01)00302-4
2. Aziz, S.A., Wang, X., Hawbani, A., Qureshi, B., Alsamhi, S.H., Alabsi, A., Zhao, L., Al-Dubai, A., Ismail, A.S.: Wireless rechargeable sensor networks: Energy provisioning technologies, charging scheduling schemes, and challenges. *IEEE Transactions on Sustainable Computing* **10**(5), 873–890 (2025). DOI 10.1109/TSUSC.2025.3549414
3. Bengio, Y., Louradour, J., Collobert, R., Weston, J.: Curriculum learning. In: *Proceedings of the 26th Annual International Conference on Machine Learning*, pp. 41–48 (2009). DOI 10.1145/1553374.1553380
4. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y.: Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734 (2014). DOI 10.3115/v1/D14-1179
5. Comaniciu, D., Meer, P.: Mean shift: A robust approach toward feature space analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **24**(5), 603–619 (2002). DOI 10.1109/34.1000236
6. van Hasselt, H., Guez, A., Silver, D.: Deep reinforcement learning with double Q-learning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, pp. 2094–2100 (2016). DOI 10.1609/aaai.v30i1.10295
7. Hausknecht, M., Stone, P.: Deep recurrent Q-learning for partially observable MDPs. In: *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents* (2015). ArXiv:1507.06527
8. He, L., Kong, L., Gu, Y., Pan, J., Zhu, T.: Evaluating the on-demand mobile charging in wireless sensor networks. *IEEE Transactions on Mobile Computing* **14**(9), 1861–1875 (2015). DOI 10.1109/TMC.2014.2368557
9. He, S., Chen, J., Jiang, F., Yau, D.K.Y., Xing, G., Sun, Y.: Energy provisioning in wireless rechargeable sensor networks. *IEEE Transactions on Mobile Computing* **12**(10), 1931–1942 (2013). DOI 10.1109/TMC.2012.161
10. Heinzelman, W.R., Chandrakasan, A., Balakrishnan, H.: Energy-efficient communication protocol for wireless microsensor networks. In: *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*, vol. 2, p. 10 (2000). DOI 10.1109/HICSS.2000.926982
11. Hong, Y., Yang, Y., Luo, C., Li, D., Lu, Y., Chen, Z.: Spatiotemporal optimization for charging scheduling in wireless rechargeable sensor networks. *IEEE Internet of Things Journal* **11**(2), 3056–3067 (2024). DOI 10.1109/JIOT.2023.3294434
12. Kaswan, A., Jana, P.K., Das, S.K.: A survey on mobile charging techniques in wireless rechargeable sensor networks. *IEEE Communications Surveys & Tutorials* **24**(3), 1750–1779 (2022). DOI 10.1109/COMST.2022.3189387
13. Kurs, A., Karalis, A., Moffatt, R., Joannopoulos, J.D., Fisher, P., Soljačić, M.: Wireless power transfer via strongly coupled magnetic resonances. *Science* **317**(5834), 83–86 (2007). DOI 10.1126/science.1143254

14. Li, L., Feng, Y., Liu, N., Li, Y., Zhang, J.: Deep reinforcement learning-based dynamic charging-recycling scheme for wireless rechargeable sensor networks. *IEEE Sensors Journal* **24**(9), 15,457–15,471 (2024). DOI 10.1109/JSEN.2024.3380592
15. Lin, C., Xue, B., Wang, Z., Han, D., Deng, J., Wu, G.: DWDP: A double warning thresholds with double preemptive scheduling scheme for wireless rechargeable sensor networks. In: *Proceedings of the IEEE International Conference on High Performance Computing and Communications (HPCC)*, pp. 503–508 (2015). DOI 10.1109/HPCC-CSS-ICISS.2015.115
16. Lyu, Z., Wei, Z., Pan, J., Chen, H., Xia, C., Han, J., Shi, L.: Periodic charging planning for a mobile WCE in wireless rechargeable sensor networks based on hybrid PSO and GA algorithm. *Applied Soft Computing* **75**, 388–403 (2019). DOI 10.1016/j.asoc.2018.11.022
17. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D.: Human-level control through deep reinforcement learning. *Nature* **518**(7540), 529–533 (2015). DOI 10.1038/nature14236
18. Nguyen, T.N., Liu, B.H., Chu, S.I., Do, D.T., Nguyen, T.D.: WRSNs: Toward an efficient scheduling for mobile chargers. *IEEE Sensors Journal* **20**(12), 6753–6761 (2020). DOI 10.1109/JSEN.2020.2974255
19. Rashid, T., Samvelyan, M., Schroeder de Witt, C., Farquhar, G., Foerster, J., Whiteson, S.: QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In: *Proceedings of the 35th International Conference on Machine Learning, Proceedings of Machine Learning Research*, vol. 80, pp. 4295–4304 (2018)
20. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*, 2nd edn. MIT Press, Cambridge, MA (2018)
21. Tomar, A., Muduli, L., Jana, P.K.: A fuzzy logic-based on-demand charging algorithm for wireless rechargeable sensor networks with multiple chargers. *IEEE Transactions on Mobile Computing* **20**(9), 2715–2727 (2021). DOI 10.1109/TMC.2020.2990419
22. Wang, K.: Dynamic on-demand collaborative charging scheduling based on game theory. *IEEE Journal of the Electron Devices Society* **12**, 824–830 (2024). DOI 10.1109/JEDS.2023.3342720
23. Wedaj, F.T., Hawbani, A., Wang, X., Qaisar, M.U.F., Othman, W., Alsamhi, S.H., Zhao, L.: RECO: On-demand recharging and data collection for wireless rechargeable sensor networks. *IEEE Transactions on Green Communications and Networking* **7**(4), 1863–1876 (2023). DOI 10.1109/TGCN.2023.3305026
24. Xie, L., Shi, Y., Hou, Y.T., Sherali, H.D.: Making sensor networks immortal: An energy-renewal approach with wireless power transfer. *IEEE/ACM Transactions on Networking* **20**(6), 1748–1761 (2012). DOI 10.1109/TNET.2012.2185831
25. Xu, Q.C.: Efficient collaboration between multiple mobile chargers in wireless rechargeable sensor networks. Master’s thesis, National Chung Hsing University (2025)
26. Yang, M., Liu, N., Zuo, L., Feng, Y., Liu, M., Gong, H., Liu, M.: Dynamic charging scheme problem with actor-critic reinforcement learning. *IEEE Internet of Things Journal* **8**(1), 370–380 (2021). DOI 10.1109/JIOT.2020.3005598